UniTrade — Software Architecture Specifications (SAS)

3.2.1 Architectural Requirements

Three concerns drive Unitrade's architecture. The platform has to act as a trusted intermediary trip between students who don't know each other, it has to work reliably on mobile devices over limited connectivity, and it has to remain maintainable by a small team working across separate domains in parallel. The patterns described below follow from those concerns.

Architectural Patterns

Client-server (system-wide).
The system is strictly client-server. A single thin client handles presentation and user interaction; all business logic, data storage, verification, payment confirmation, and moderation reside on the server. Clients cannot communicate directly with one another or bypass the server.

This is a deliberate choice rather than a default. UniTrade facilitates peer-to-peer exchange in the physical world — buyers and sellers meet in person — but the platform itself remains centralised, because verified identity, audit trails, and dispute resolution all require a trusted authority. A peer-to-peer architecture at the system level would undermine every trust guarantee the platform is built on. The accepted trade-off is that all transactional operations require server connectivity, there is no offline-first capability.

Layered (n-tier) architecture.
Five layers, each depending on the layer below it:

Layer	Responsibility
Presentation	Client UI and user input only
Edge	TLS termination, routing, and protection
API	Request validation, authorisation, routing — no business logic
Application	Core business logic, organised into domain modules
Data	Relational storage, cache, and search

The separation keeps business rules independent of transport concerns: changing a route does not require touching domain logic, and changing an external provider does not require touching a controller. The cost is additional indirection relative to placing logic directly in controllers — accepted because the alternative does not scale to multiple developers working concurrently.

Modular Monolith (Application layer).
All business domains are deployed as a single application but separated internally into modules: Identity, Listings, Reservations, Transactions, Chat, Reviews, Wishlist, and Notifications, Reference Data, Reputation, Disputes, and Audit.

Microservices were considered and rejected. The domains are tightly coupled by nature — creating a reservation atomically touches Listings, Reservations, Chat, Wishlist, and Notifications — and distributing that operation would introduce distributed transactions, inter-service latency, and operational overhead disproportionate to the team's size. The accepted trade-off is that modules cannot be scaled independently; the system scales as a whole. Because module boundaries are enforced in code, extracting a module later remains a refactor rather than a rewrite.

Domain-Driven Design (module boundaries).
Each module constitutes a bounded context with its own models, repositories, and service interface and exception type. Modules interact through defined interfaces rather than reaching into one another's internal data. This is what makes the modular monolith genuinely modular rather than a folder convention. Some duplication of similar DTOs across boundaries is accepted deliberately as shared models are the primary mechanism by which bounded contexts erode.

State machine (Reservations module).
The reservation lifecycle is governed by centralised guarded transitions rather than status checks distributed across controllers, workers, and handlers. States progress from active to completed, expired, or cancelled, with each transition validated in one place and driven by either user action or system event. Keeping the lifecycle rules in a single location makes every transition predictable and auditable.

Communication patterns.
The system uses four distinct communication mechanisms selected per interaction type:

- **Request-response (primary) —** REST over HTTPS for browsing, registration, reservations, and payment initiation. The API is naturally resource-oriented, which maps cleanly onto REST semantics.

- **Push (real-time) —** WebSockets via SignalR for chat, reservation status changes, and payment events, avoiding the latency and load of client polling. SignalR does not queue messages for disconnected clients, so any flow depending on real-time delivery is paired with a fallback query.

- **Inbound webhooks (event-driven) —** the payment gateway notifies the server directly when a payment completes. This notification is a signature-verified and treated as the sole source of truth; the browser redirect that accompanies it is used only for user experience and confirms nothing, as it is client-controlled and therefore forgeable.

- **Message brokering (asynchronous) —** background work that need not block a user request is decoupled through a message queue consumed by dedicated workers.

Event-driven processing (asynchronous layer).
The application publishes events rather than invoking background work directly, and workers subscribe to the event types relevant to them. This keeps user-facing request paths free of slow operations and allows new asynchronous behaviour to be added without modifying existing logic.

Design patterns

Currently implemented:

- **Repository —** Every module exercises persistence through a repository interface rather than the data context directly, which isolates domain logic from Entity Framework and makes modules testable in isolation
- **Observer** — The real-time client maintains a subscriber collection for each event type, notifying all registered listeners when an event arrives. Server-side notifier interfaces follow the same shape

- **Dependency Inversion** — Modules depend on interfaces registered in the composition root, not concrete implementations which is what permits infrastructure to be substituted without changes to business logic

Planned, dependent on subsystems scoped for later delivery:

- **Strategy** — routing of AI verification outcomes (auto-approve, reduce visibility, escalate to review) based on confidence scoring.

- **Chain of Responsibility** — the listing submission pipeline, passing a submission through validation, scoring, moderation, and notification in sequence.

- **Template Method** — dispute resolution, where dispute types share a common workflow but apply type-specific evaluation.

- **Command** — administrative moderation actions treated as discrete, logged, reversible operations.

- **Decorator** — visibility modifiers applied to medium-risk listings without altering the listing object itself.

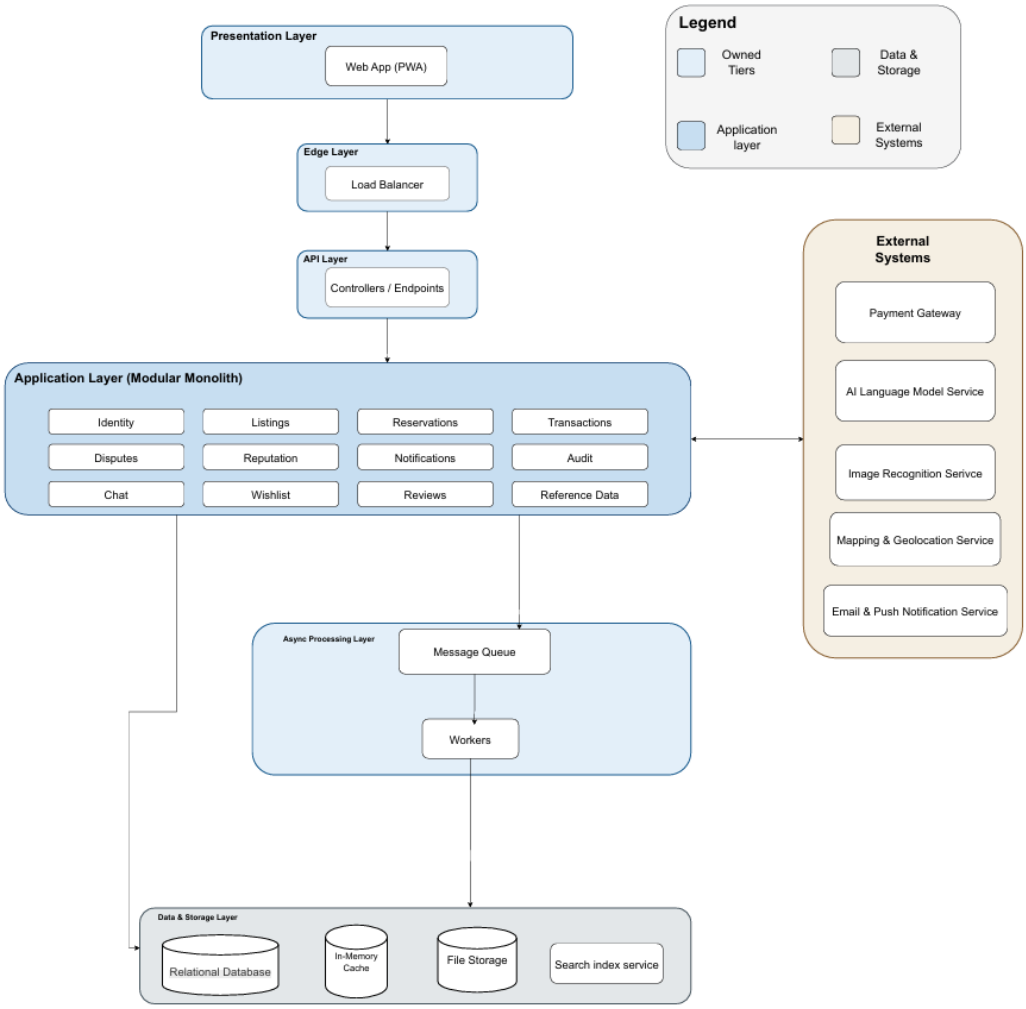
- **Saga** — long-running reservation workflows with branching and compensating actions, once orchestrated asynchronous processing is in place.

Constraints

1. **Payment gateway dependency.** The system uses PayFast as its sole payment gateway. The architecture must expose a publicly reachable webhook endpoint, treat redirect handling as separate from transaction verification, and store no card or banking credentials.
2. **University email validation.** Only students at recognised South African universities may register. Email domains are validated against the maintainable list and the domain determines institutional affiliation.
3. **PDF-only proof of registration.** The document verification pipeline is designed for PDF input.
4. **AI is advisory only.** Automated scoring may approve, reduce visibility, or escalate, but may never issue a final rejection. Rejection remains a human decision.
5. **Administrative review SLA.** The system must maintain a visible, auditable moderation queue allowing flagged cases to be resolved within the defined service window.
6. **Optional location permissions.** Meetup verification may use location data, but users cannot be compelled to grant access. The architecture must support both verified and unverified check-ins.
7. **No refunds.** Payment states are forward-only. Buyer protection is provided by physical inspection prior to payment; disputes are handled reputationally rather than through transaction reversal.
8. **Mobile-first, PWA-based delivery.** The client is a single responsive Progressive Web App prioritising low-bandwidth operation and installability without app-store distribution.
9. **Data minimisation and POPIA compliance.** Only essential user and transaction data may be stored and personal information must be protected in line with the Protection of Personal Information Act.
10. **Evolving external dependencies.** Constraints are subject to revision if gateway APIs, regulatory requirements, or the recognised institution lists change.

Architecture diagram

The diagram is organised into five architectural layers described above, with two additional groupings: an asynchronous processing region containing the message queue and its worker consumers, and an external systems region containing all third-party dependencies — payment gateway, mapping and geolocation services, notification delivery, and AI services.


Legend
Owned Tiers
Data & Storage
Application layer
External Systems
Presentation Layer
Web App (PWA)
Edge Layer
Load Balancer
API Layer
Controllers / Endpoints
Application Layer (Modular Monolith)
Identity
Listings
Reservations
Transactions
Disputes
Reputation
Notifications
Audit
Chat
Wishlist
Reviews
Reference Data
External Systems
Payment Gateway
AI Language Model Service
Image Recognition Serivce
Mapping & Geolocation Service
Email & Push Notification Service
Async Processing Layer
Message Queue
Workers
Data & Storage Layer
Relational Database
In-Memory Cache
File Storage
Search index service

Mapping Quality Requirements to Architectural Decisions

The SRS defines seven quality requirements. Each is mapped below to the architectural mechanisms intended to satisfy it.

Quality Requirement	Architectural Decision
Performance — responses within 2s for 95% of requests; chat delivery within 500ms	Composite and filtered database indexes targeting high-frequency query paths (listing feed, course and category browse); a distributed cache layer for session and frequently-read data; a push-based real-time channel that eliminates client polling; asynchronous offloading of non-blocking work to background workers.
Scalability — 200% workload growth with no more than 10% performance degradation and no major re-architecture	A stateless API layer permitting horizontal scale-out behind edge routing, container-based deployment with per-revision scaling; enforced module boundaries allowing individual bounded contexts to be extracted into independently scaled services without redesign, connection pooling and cache offloading to reduce database contention under load.
Security — AES-256 encryption at rest, TLS in transit, MFA for administrative accounts	Token-based authentication with audience-scoped credentials, separating general API access from real-time channel access; peppered hashing of one-time credentials so that database compromise alone does not expose them; constant-time comparison for credential verification; signature verification of all inbound payment

	notifications prior to any state change; delegation of card handling entirely to the payment gateway, removing sensitive payment data from system scope.
Reliability — continuous availability excluding scheduled maintenance; rapid recovery from failure	Blue-green deployment in which a new revision is health-checked before receiving traffic and the prior revision remains live until the new one is verified, with automatic rollback on failure; idempotent webhook handling that tolerates duplicate delivery without duplicate side effects; per-cycle failure isolation in background workers so that a single failed cycle is logged without terminating the worker; managed database backups and platform-level health monitoring with automatic restart.
Maintainability — rapid deployment turnaround and sustained automated test coverage	Modular monolith with domain-aligned boundaries confining most changes to a single module; automated CI enforcing build, test, formatting, and static analysis before merge; automated deployment with database migrations applied as part of the release pipeline; separated unit and integration test suites, the latter executing against a real database engine rather than an in-memory substitute; centralised structured logging and monitoring.
Usability — core tasks completable within 5 minutes; accessible across devices	A single Progressive Web App client, eliminating divergence between web and mobile experiences; real-time updates removing the need for manual refresh during coordination and payment; an interactive map interface with text-based search as an alternative input path for users who cannot interact with map controls.
Administration — verification and dispute resolution within defined service windows	A persisted verification request entity with an explicit status lifecycle, providing a queryable and auditable moderation queue ordered by submission age; notification dispatch on every status-changing decision; audit logging of administrative actions with actor, timestamp, and reason retained for compliance.

3.2.2 Technology Requirements

Frontend(Web & Mobile) :react + Vite(PWA):

We chose React as the foundation of Unitrade's client because it directly supports the project's core goals of scalability, usability, and efficient development. React's component-based architecture is well suited to building interactive, modular interfaces such as product listings, carts, messaging, and notifications, allowing features to be developed, tested, and updated independently. We build with **Vite** as the build tool for its fast development server and build times, and we ship the client as a **Progressive Web App** using **vite-plugin-pwa**, which gives us installability , offline support, and push notifications on mobile devices without maintaining a separate native codebase.

Service worker architecture: the PWA uses a single shared service worker (firebase-messaging-sw.js), generated via injectManifest rather than the vite-plugin-pwa default of generateSW. This was a deliberate architectural choice as it lets us hand-write the Firebase Cloud Messaging(FCM) push-handling logic while still having vite-plugin-pwa inject the PWA precache manifest into the same file, so that FCM push notifications and PWA asset precaching coexist in a single service worker instead of two competing ones.

Backend API: ASP.NET Core(C#):

We use ASP.NET Core because it provides a secure, scalable, cloud-ready backend that fits naturally with our Azure hosting environment, simplifying deployment, scaling, and monitoring. Authentication and authorization are handled via JWT Bearer Authentication, giving us stateless, standards-based security with minimal custom code, which supports POPIA compliance. Overall, ASP.NET Core gives us reliable security, scalability, and efficient development for a growing student marketplace.

Primary Database : PostgreSql:

We use PostgreSQL as our primary database. Our data is highly structured and relational, requiring strong consistency and enforced relationships to meet POPIA requirements, and Postgres provides this along with encryption, backup, and scalability support. PostgreSQL also provides a strong foundation for AI-related capabilities we plan to build later, such as vector search, though this is future scope and not part of the current implementation.

Caching: Azure Cache for Redis

Azure Cache for Redis is used as a high-speed caching layer for frequently accessed data, including listings and user profiles. This reduces load on the primary database and improves response times for the data users request most often.

Realtime Communication: SignalR

SignalR powers real-time communication across the application. This includes live chat between users, reservation updates, payment/PIN notifications, meetup updates, and other real-time user notifications, so users see changes as they happen rather than needing to refresh or poll for updates.

Mapping:Leaflet + OpenStreetMap + Nominatim:

We use Leaflet as our map rendering library, backed by OpenStreetMap map data and Nominatim for geocoding, to let users choose a location within a radius of where they currently are in order to propose a meet-up point for an exchange. This avoids the licensing and usage costs of a commercial mapping provider while giving us the location-picking functionality the marketplace needs.

Hosting and cloud infrastructure : Microsoft azure :

We host Unitrade on Microsoft Azure to provide a scalable, reliable, and low-maintenance platform. Azure Container Apps hosts and runs the application, pulling its container images from Azure Container Registry. The application stores its relational data in Azure Database for PostgreSQL, and offloads asynchronous and background processing ,such as reservation expiry timers and background notification jobs ,to Azure Service Bus, keeping these tasks off the main request path. Transactional email is sent through Azure Communication Services, and Azure Cache for Redis sits alongside these services as the shared caching layer. Azure Front Door sits in front of the application, providing CDN and edge routing, which we've included given the emphasis placed on performance, availability, and security for this project. Managed Identity lets Azure services authenticate to each other without storing credentials in code or configuration, Log Analytics provides monitoring and diagnostics across these resources, and Resource Groups are provisioned per environment to keep environments isolated from one another.

CI/CD: GitHub Actions automatically builds, tests, and deploys the application, with deployments to Azure Container Apps using a blue-green strategy. The full pipeline is covered in detail in the separate deployment document.

Payments: PayFast:

We use PayFast for payment processing. PayFast provides a fully featured developer sandbox with test credentials, letting us build and test the complete payment flow without waiting on business/merchant verification which is a good fit for a project still in active development. Payment confirmation is handled via PayFast's signed ITN (Instant Transaction Notification) webhook, rather than relying on the browser redirect after checkout, so a payment is only considered confirmed once we've received and verified a signed server-to-server notification from PayFast directly.

Notifications: FCM + Azure Communication Services

We use a multi-channel notification system, so students are reliably informed :

- **FireBase Cloud Messaging (FCM):** delivers instant push notification to the PWA
- **Azure Communication Services:** sends transactional emails (see Hosting and clous infrastructure above).

Testing Stack:

- **Backend:** xUnit for test authoring, with Coverlet for code coverage collection.
- **Frontend:** Vitest for test authoring, with v8 for code coverage collection
- **End-to-end**: Playwright, exercising the application through a real or staged running instance.
- **Static analysis:** SonarCloud.

Secrets Management:

Local development uses local configuration files. Deployment secrets are stored in GitHub Encrypted Secrets and are injected into the deployment pipeline and application configuration at deployment time. Azure Key Vault is intentionally not part of the current architecture. We decided to do this as it was made on the basis that Key Vault would be overkill relative to the project's current scale and requirements.

Notes on Key Architectural Changes

The sections above describe our current technology stack. A few changes from earlier decisions are worth documenting , since they explain why the current architecture looks the way it does:

- **Resend → Azure Communication Services:** we moved away from Resend because, without a registered sending domain, emails could only be sent to our own verified addresses. Azure Communication Services allows us to send transactional emails to real users while fitting naturally into the rest of our Azure infrastructure.
- **MSSQL→ PostgreSQL:** PostgreSQL is a better fit for the project's future roadmap, particularly its support for AI-related capabilities such as vector extensions, while still providing the strong relational features the system requires today.
- **Ozow → PayFast:** Ozow requires a registered business before developer credentials are issued, which wasn't practical for development. PayFast provides a fully featured sandbox environment, better developer support, and let the team build and test the complete payment flow without waiting for business verification.

Deployment Requirements:

-Live, Accessible System:

https://ca-frontend-prod.kindgrass-55a2ae94.southafricanorth.azurecontainerapps.io

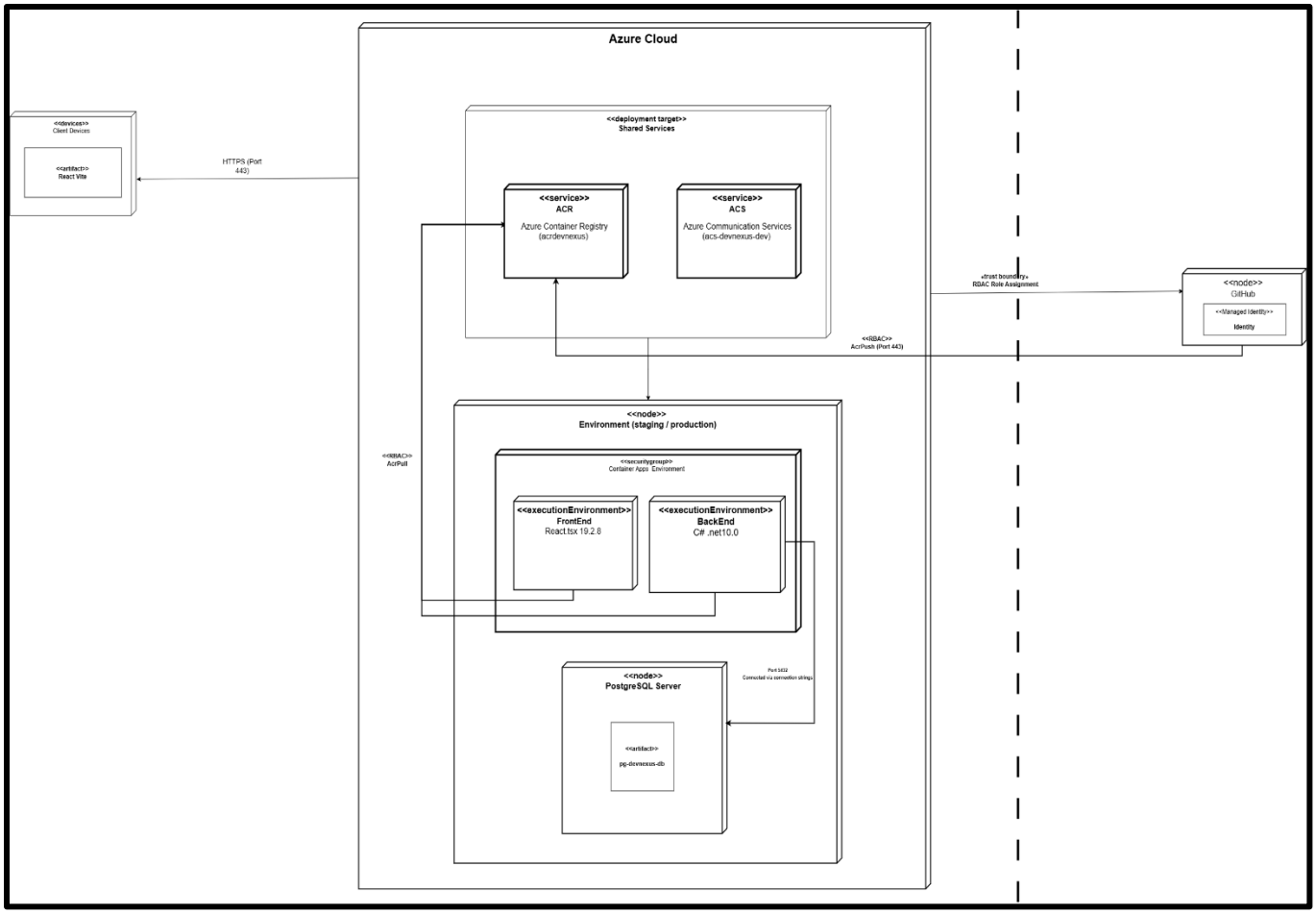
-Environment Parity: The system distinguishes between dev, staging, and production environments, each with its onw azure container apps environment, PostgreSQL flexible Server, and resource group. All three share a single azure container registry. The main branch triggers an automated deployment to staging. Followed by a manual approval gate ,that is approved by the devOps and team lead only , before promoting the same image to production.

-**Infrastructure as Code (IaC**) :

The system uses Azure Bicep templates to declaratively define and deploy all infrastructure-container apps environment, backend and frontend Container Apps, Azure Container Registry, PostgreSQL Flexible Server, Communication Services, log analytics, and application insights. Deployments are executed via `az deployment sub create` against modular Bicep templates (main.bicep+per source modules), parameterized per environment.

-Secrets Management: Credentials, connection strings, and API keys are never committed to the repository. All our secrets (ACR credentials, Postgres connection strings, JWT/otp signing secrets, Firebase config, Azure Communication Services connection string, and Azure OIDC identity values) are stored as encrypted Github Actions secrets and injected at deploy time via `az containerapp secret set` and `--set-env-vars`, referenced in the container app as `secretref:` values rather than plaintext. Azure authentication for the ci/cd pipeline uses openID connect (OIDC) via a user-assigned managed identity with federated credentials scoped to specific Github environments(staging ,production), no long -lived azure credentials are stored in github

-Rollback Strategy: Blue-green is used as our rollback strategy. Each deploy creates a new “green” container revision alongside the existing “blue” one. Traffic is shifted to green only after a health check succeeds. If deployment or the health check fails, the pipeline automatically shifts traffic back to the last know-good revision and deactivates the failed one.



CI/CD flow diagram :